Introduction to Deep Learning
Assignment 1: building MLPs, CNNs and generative models with TensorFlow

Myriana Miltiadous(3699463), Pelagia Kalpakidou(3744825), Elena Pitta(3840220)

Introduction

Deep learning has influenced artificial intelligence by allowing computers to learn complicated patterns and representations from large datasets. This report delves into deep neural networks by combining the Keras and TensorFlow libraries. The challenges of creating multi-layer perceptrons (MLPs), convolutional neural networks (CNNs), and generative models are addressed through a series of tasks that are going to be explained and evaluated.

Task 1: Learn the basics of Keras API for TensorFlow.

This task's objective was to develop proficiency in using the open-source libraries Keras and TensorFlow. The examined deep neural network architectures were Multi-Layer Perceptrons (MLPs) and Convolutional Neural Networks (CNNs). Different configurations were investicated, in order to understand how hyperparameter adjustment impacts the model's performance.

1.1 Methodology

1.1.1 Experimenting with provided resources

To begin with, two major examples were taken from an official repository[1]: mnist_mlp.py and mnist_cnn.py. These scripts demonstrate the training and evaluation of a basic Multilayer Perceptron (MLP) and Convolutional Neural Network (CNN), respectively, on the MNIST dataset. Exploring these instances was critical in understanding the core complexities of neural network parameters and their respective reasonable ranges, laying a solid foundation for exploring with more complex datasets.

1.1.2 Train various MLPs and CNNs on Fashion MNIST and CIFAR-10 datasets

After gaining insights from experimenting with the aforementioned scripts and studying neural network documentation, experiments were conducted using Fashion MNIST and CIFAR-10 datasets. The first dataset contains 2D (grayscale) images of size 28x28 split into 10 categories that represent types of clothes. From this dataset, 60,000 images were used for training and 10,000 for testing. The second dataset contains 32x32x3 RGB images for 10 classes that represent animals or modes of transportation. From this dataset, 50,000 images were used for training and 10,000 for testing.
The objective was to explore diverse configurations of Multilayer Perceptrons (MLPs) and Convolutional Neural Networks (CNNs) using the fashion MNIST dataset, which is comparatively simpler than CIFAR-10. To optimize hyperparameters, 10% of the training data was reserved for the validation set.
Subsequently, the top three sets of hyperparameters that yielded the best performance were chosen. These sets were then used to train the respective models on the CIFAR-10 dataset, enabling a comparison of their performance on both CIFAR-10 and fashion MNIST datasets.

1.1.2.1 Fashion MNIST In order to discover the models with the best performance on the Fashion MNIST dataset the Keras Tuner library, which is a hyperparameter tuning tool for Keras models, was utilised. It was chosen since it provides various tuning strategies. For using it, a function that constructs and compiles the model was created. This function takes as input a `kt.HyperParameters` object (a class representing a collection of hyperparameters that can be tuned during the hyperparameter search process), allowing the definition of hyperparameters (integers, floats, strings, etc.) and their possible value ranges. The specified hyperparameters can then be used to build and compile the model, enabling optimization.

1.1.2.1.1 MLP The implemented function for exploring MLPs went through a thorough experimentation process, testing a wide range of combinations. These combinations included various hidden layer numbers (1 to 8-each followed by a ReLU activation function), neurons per layer (16 to 256 with a step size of 16), learning rates (logarithmically sampled between 1e-4 and 1e-2), optimizers ("sgd" or "adam"), batch sizes (16, 32 or 64), regularisation approaches (1e-6 and 1e-2), and dropout rates (0.1 to 0.7 with increments of 0.1). For the output layer the softmax activation function was used,

[1] `https://github.com/keras-team/keras/tree/tf-keras-2/examples/`

which is appropriate for multiclass classification since it transforms neuron outputs into probabilities (one for every class), ensuring their sum is 1. The Hyperband Tuner, a powerful optimization tool that dynamically allocates resources based on the performance of alternative configurations, was used to intelligently determine the architecture and parameters for each MLP. This strategic approach meant that computational resources were employed efficiently, allowing for focused exploration of the hyperparameter space and, eventually, the identification of the 3 MLP structures with the best performances. The top three models that were found had the following configurations:
Top 1 Model: hidden layers: 4, neurons per layer: 160, learning rate: 0.0005594907608422082, optimizer: 'adam', batch size: 32, dropout rate: 0.1 with.
Top 2 Model: hidden layers: 6, neurons per layer: 112, learning rate: 0.0004426251236616376, optimizer: 'adam', batch size: 32, dropout rate: 0.4.
Top 3 Model: hidden layers: 7, neurons per layer: 96, learning rate: 0.00044489082371598814, optimizer: 'adam', batch size: 32, dropout rate: 0.6.

1.1.2.1.2 CNN For optimizing the CNNs, the created function for exploring the different configurations was quite different than the one for MLPs. The model constructed in the function consists of two convolutional layers, each followed by a ReLU activation function. The number of filters in the first and second convolutional layer, are explored between 32 to 128 (with a step of 16) and 32 to 64 (with a step of 16) respectively. Additionally, the two respective kernel sizes, are chosen from the options 3 and 5. Moreover, the model has one additional dense layer before the output layer (with 10 neurons-one per class and softmax activation function). The number of its units is again explored from 32 to 128 with a step of 16. Finally, The Adam optimizer is utilized with a learning rate chosen from the options [1e-2, 1e-3].
The top three models that emerged have the following configurations:
Top 1 Model: First convolutional layer: 112 filters, (3,3) kernel size. Second convolutional layer: 64 filters, (3,3) kernel size. First dense layer: 48 units, learning rate: 0.001.
Top 2 Model: First convolutional layer: 48 filters, (3,3) kernel size. Second convolutional layer: 64 filters, (5,5) kernel size. First dense layer: 96 units, learning rate: 0.001.
Top 3 Model: First convolutional layer: 112 filters, (3,3) kernel size. Second convolutional layer: 32 filters, (5,5) kernel size. First dense layer: 128 units, learning rate: 0.001.

1.1.2.2 CIFAR-10

1.1.2.2.1 MLP The same reported best three MLPs were trained and evaluated on the CIFAR-10 dataset.

1.1.2.2.2 CNN The same reported best three CNNs were trained and evaluated on the CIFAR-10 dataset.

1.2 Experiments and Results

1.2.1 MLPs

The reported MLPs given by the hyperband tuner were again trained on the Fashion MNIST dataset for 100 epochs, in order to visualize their accuracy and loss function (categorical cross-entropy) and understand better their performance. A dropout rate of 0.2 was additionally added after each layer in order to prevent overfitting.

1.2.1.1 Fashion MNIST The accuracy and loss of MLPs on Fashion MNIST was finally calculated on the test set as presented below for each model:
Model 1: Test loss: 0.3948231339454651 Test accuracy: 0.88919997215271
Model 2: Test loss: 0.4106953740119934 Test accuracy: 0.8935999870300293
Model 3: Test loss: 0.3748060166835785 Test accuracy: 0.89120000600814825

As shown in figure 1, accuracy of the MLPs trained on Fashion MNIST increased steadily with similar way in all the three models, while training loss decreased. However, the validation curves are not as close to the training curves, especially for the loss, which shows that there's a little bit of overfitting, which can be addressed by increasing dropout or apply regularization. However, this was not considered necessary since, the final evaluation on the test data yields a commendable accuracy of more than 88% and a test loss of less than 0.42 in all the models, which was considered accepted. That demonstrated the effectiveness of the models in achieving high performance on previously unseen examples.

1.2.1.2 CIFAR-10 Again for CIFAR-10 the accuracy and loss of MLPs was calculated on the test set as presented below for each model:
Model 1: Test loss: 1.6269999742507935 Test accuracy: 0.41990000009536743
Model 2: Test loss: 1.5361874103546143 Test accuracy: 0.4505999982357025

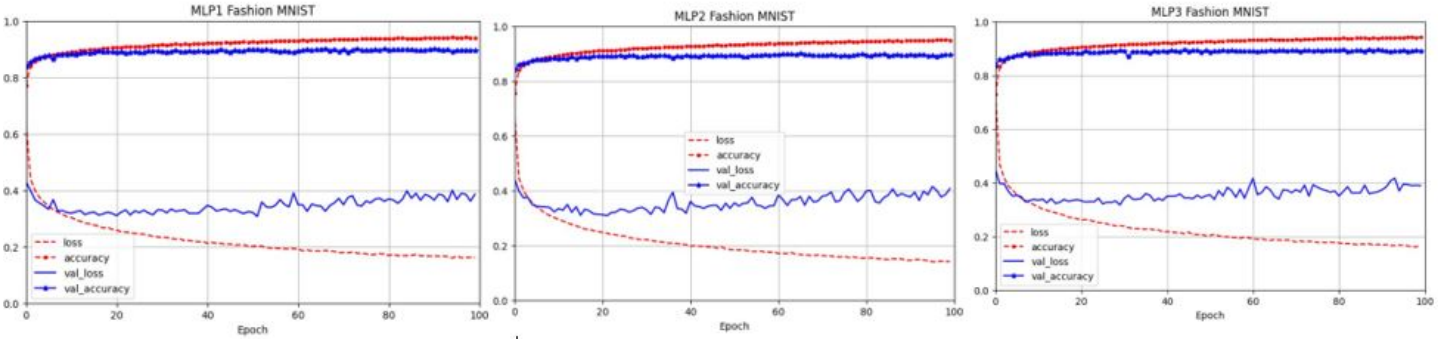


Figure 1: Three best performed MLPs on Fashion MNIST

Model 3: Test loss: 1.5531246662139893 Test accuracy: 0.44510000944137573
From the figure 2 we can see that the validation metrics, this time, are relatively close to the training metrics and the overfitting concern is less pronounced, suggesting that model is generalizing to unseen data. However, it is obvious that compared to the performance of the specific MLPs on the Fashion MNIST dataset, for CIFAR-10 they have performed signifficanly worse.

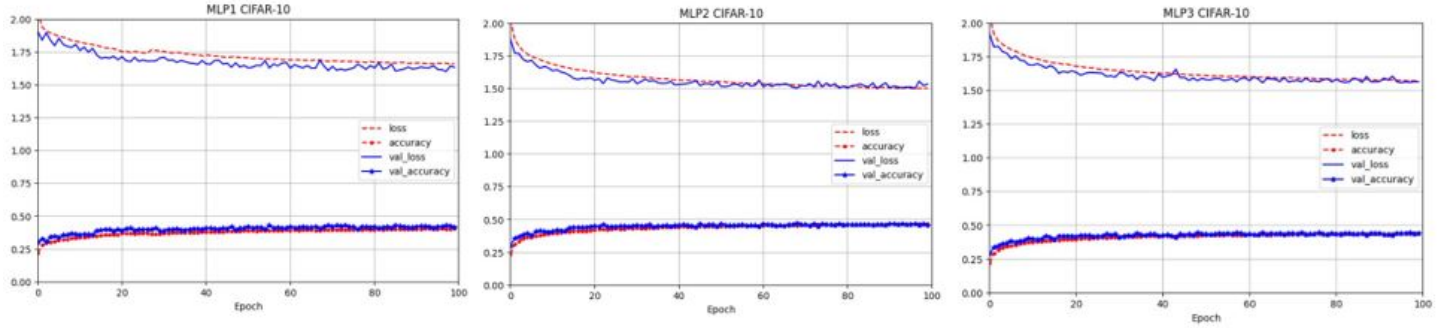


Figure 2: MLPs trained on CIFAR-10

1.2.2 CNNs The reported CNNs given by the hyperband tuner were again trained on the Fashion MNIST dataset for 25 epochs, in order to visualize their accuracy and loss function (categorical cross-entropy) and understand better their performance, as was done with the MLPs.

1.2.2.1 Fashion MNIST

The accuracy and loss of CNNs on Fashion MNIST was also calculated on the test set as presented below for each model:
Model 1: Test loss: 0.28063440322875977 Test accuracy: 0.9221000075340271
Model 2: Test loss: 0.23696838319301605 Test accuracy: 0.9247999787330627
Model 3: Test loss: 0.2756352424621582 Test accuracy: 0.9225999712944031
As once again shown in the figure 3, accuracy of the CNNs trained on Fashion MNIST increased steadily with similar way in all the three models, while training loss decreased. However, as in the case of MLPs the validation curves are not as close to the training curves, especially for the loss, which shows that there's a little bit of overfitting again. Nonetheless, the final evaluation on the test data yields a significant accuracy of more than 92% and a test loss of less than 0.29 in all the models, which was considered enough for concluding to this final models, as the top 3.

1.2.2.2 CIFAR-10 As before, the same CNNs were trained on CIFAR-10 and their accuracy and loss was calculated on the test set as presented below for each model:
Model 1: Test loss: 1.4000815153121948 Test accuracy: 0.6064000129699707
Model 2: Test loss: 1.1682028770446777 Test accuracy: 0.6897000074386597
Model 3: Test loss: 1.1092770099639893 Test accuracy: 0.6977999806404114

From the figure 4, similar trends can be observed in the way the validation and training accuracy and loss of the three models vary over time. They demonstrate a steady decrease in training loss and rise in training accuracy, reaching a peak of about 88%. But a significant discrepancy in the accuracies and losses between training and validation data indicates that

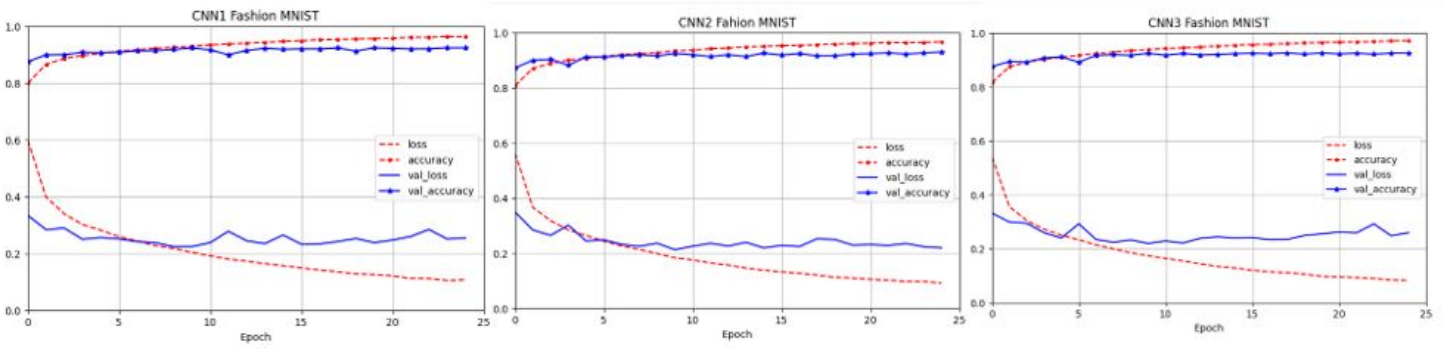


Figure 3: CNNs trained on Fashion MNIST

overfitting is presented. Especially the fluctuating validation loss, which is away from training loss, suggests the need for further fine-tuning for the model to be able to achieve higher test accuracies.

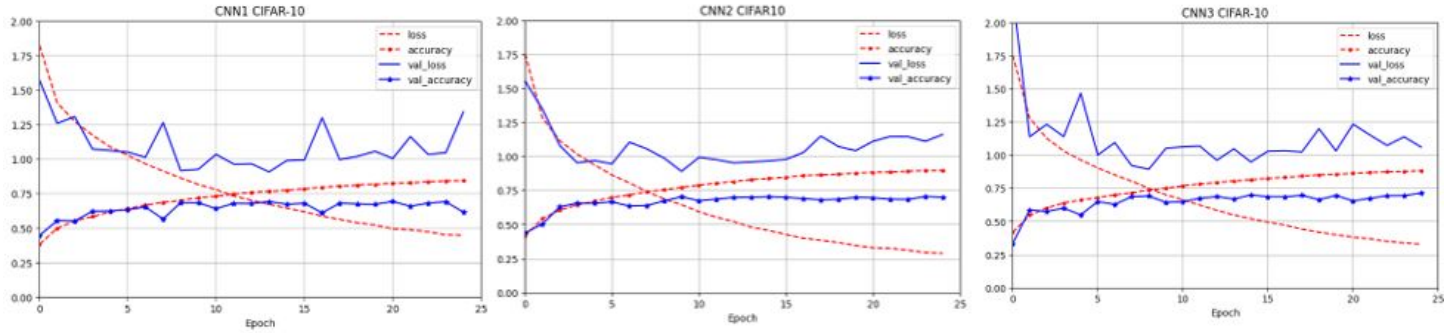


Figure 4: CNNs trained on CIFAR-10

1.3 Discussion

That big difference in performance of the models between the two datasets can be explained by several reasons. Firstly, the CIFAR-10 dataset contains more complex images as they are colourful and they represent both objects and animals, whereas the fashion MNIST consists of grayscale images of clothes. Consequently, a more complex model is more likely to achieve handling the diverse images of CIFAR-10. Moreover, even if some hyperparameters work well on Fashion MNIST dataset, this does not mean that they are also the best ones for CIFAR-10. Those reasons make the classification task of CIFAR-10 more challenging than the classification task of Fashion MNIST for a model, ending up with a poorer performance. In summary, all the conducted tests demonstrated the usefulness and importance of validation sets in refining hyperparameters and reducing overfitting as well as the importance of correctly chosen architectures.

Task 2: Develop a "Tell-the-time" network.

In the current task, we strive to develop and train CNN models that would give us the labels corresponding to the time shown in images of analog clocks as accurately as possible. The data set encompasses 18,000 grayscale images, each sized at 75 x 75 pixels and paired with labels indicating the hour and minute displayed on the clock face. One particularity of this task that increases its complexity is the different angle positions and the light reflection from within the scene for each image. Our approach involves different representations of the labels varying for each of the three chosen formulations of the problem. Initially, we approach it as a multi-class classification task, followed by a regression problem, and ultimately as a multiple output heads model that combines the two strategies. With this comprehensive approach, we aim to examine how the different architectures and the adaptations at the output layer of our neural networks affect the training time and performance. Before starting, we maintained a ratio of 80:20% for the train and test sets, respectively, to guarantee a more robust evaluation of the performance of our CNN models. Additionally, we used a validation set to improve the reliability of the models, enabling monitoring and fine-tuning throughout the training process.

2.1 Methodology

2.1.1 Classification

Our first model involved approaching the problem as a multi-class classification, starting with a more manageable number of categories (n-classes) and sequentially increasing this number by employing finer intervals for sample grouping. Within the classification framework, the goal is to accurately assign each image to the corresponding category aligned with its original label after establishing categories that represent distinct time periods (for instance, in half-hour increments). We initiated the process with 24, 72, and 720 categories, which correspond to half-hour intervals, ten-minute intervals, and one category for each hour-minute combination. Our multi-class CNN models' architectural design remains generally consistent. Even so, the increasing number of categories required adjustments and awareness of the overfitting problem.

We started with a 24-class CNN architecture with three convolutional layers (with 3×3 kernels), followed by a Rectified Linear Unit (ReLU) layer and a pooling layer. The convolutional layers gradually reduce image size while increasing depth. Notably, the number of filters decreases between layers, falling from 128 to 32 to extract increasing information in deeper layers. To stabilize and accelerate the training process, batch normalization is selectively done after the initial and final convolutional layers. Following each convolutional layer, a max pooling layer (with a 2 x 2 kernel - that divides each spatial dimension by a factor of two) keeps only the strongest features while discarding all the unimportant ones, providing a cleaner output for the following layers to work with and greater translation invariance. A flattened layer reshapes the output of the convolutional layers into a 1D vector, preparing it for processing through dense layers. Following flattening, a regular feedforward neural network with two dense fully connected layers (ReLU) and a dropout rate of 0.4 for regularization. Finally, the final output layer comprises 24 neurons (estimated class probabilities), aligning with the 24-time categories. This final layer employs the softmax activation.

Subsequently, we constructed the 72-class model based on the structure of the 24-class CNN architecture. The 72-category model starts with a convolutional layer that contains 128 filters and 3x3 kernels and is generated by Rectified Linear Units (ReLU). This process repeats with two additional convolutional layers with 64 and 32 filters, each followed by max-pooling with a 2x2 kernel and batch normalization. After flattening, two dense fully connected layers, each with ReLU activation, were applied. However, in order to reduce overfitting, we increased the dropout rate to 0.5 so that the accuracy values for the training and validation set would be close. The final output layer comprises 72 neurons, corresponding to the 72-time categories for 10-minute intervals.

Finally, for our last multi-class classification model with CNNs, we tried to train a model as robust and effective as possible, but that was a great challenge. Having all 720 labels as unique categories, increased the complexity of the problem and the overfitting concerns. To overcome some of these drawbacks, we increased the depth of the model we used for the two previous n-classes problems. More specifically, we added a convolutional layer so that we have filters going from 256 to 32. The batch normalization and the maw pooling layers stayed as they were, but after flattening, we introduced three dense fully connected layers using ReLU and a dropout rate of 0.6. Despite these adjustments and the 500 epoch, we did not manage to have efficient learning progress as the accuracy in the validation set is not equivalent to the one for the training set. Hence, the challenge of training a network using all 720 categories was to find an architecture that would balance our learning goals with the overfitting problem. Moreover, the training time was another challenge as the model with all these categories was computationally heavy.

The loss function we chose for all our models was appropriate for our objective. As a result, we picked categorical cross-entropy, a metric for multi-class classification in which the target variable is one-hot encoded. Additionally, we set the Adam optimizer aiming to have an adapting learning rate, and for the model's performance evaluation during training, we used the accuracy metric, which measures the proportion of correctly classified samples. Only for our last model corresponding to the classification of the 720 categories, we customized the Adam optimizer by explicitly setting the learning rate to 0.0001.

2.1.2 Regression

Our second model involved approaching the problem as a regression problem, with categorical labels transformed into a single continuous value. To handle this transformation, the mean absolute error (MAE) loss function was employed during the model compilation. The mean absolute error loss minimizes the average absolute difference between predicted and actual time values and thus is suitable for the linear nature of the problem.

Moving on to the architecture of the regression model, we used the same structure as we did for the 24-class classification model. Hence, we have three convolutional layers and two dense layers with the difference that in that case, they result in a single output node with a linear activation function. With the linear activation function, the model can anticipate time continuously. Batch normalization and max pooling were also used in the same way as we did for the classification models.

While this representation accurately depicts the continuous nature of time, still there are some problems with it. The model may encounter difficulties dealing with time's cyclic patterns. More specifically, the linear representation may not be an efficient way to predict accurately times near the transition points on the clock face (for example the transition from 11 to 12 o'clock). To overcome this problem, we implemented the common sense error function (see section 2.1.2.1) to inspect for the inaccuracies and misinterpretations of the regression model caused by the transition of the categorical labels into a continuous scale.

2.1.2.1 Common Sense Error For the regression problem even though the loss of the model is calculated using the mean absolute error formula, it was considered necessary to implement a function that calculates the common sense error for the predictions of the model using the test set. This is because of considering cases, like "predicted" time being 11:45 (output of network=11.75) and "target" time being 0:03 (output of network=0.05), for which the loss function calculates an error of 11 hours and 42 minutes, whereas the common sense error is just 18 minutes etc. This was calculated for every prediction with the following formula:

$$error = 60 \times \min\left(|\text{predicted} - \text{real}|, 12 - |\text{predicted} - \text{real}|\right) \quad (1)$$

The formula starts by calculating the absolute difference between predicted and real values. Then, the minimum value between that absolute difference and the value of 12 after deducting that absolute difference is multiplied by 60 to be transformed in minutes and this is the final calculation of the error of a specific prediction. The overall common sense error was then calculated by taking the mean of all those errors. This approach ensures that errors can not be more than 6 hours, which is what is needed for a common sense error.

In addition to this, the errors of the two pointers (one for hour and one for minutes) of each clock was calculated in order to have more specific insights about the performance of our model. The logic behind this calculation was that if the model predicts for example 01:15 and the real time is 02:00 the model have made 15 minute error on the minutes pointer and 1 hour error on the hours pointer. However, while calculating this errors it was taken into account that if the predicted and real times have less than 30 minutes different (e.g real: 3:40, predicted: 4:00, or real: 2:55, predicted: 4:15), then the error calculated for the hour pointer is the absolute difference between the two hours, minus one (e.g for the aforementioned examples the errors for the hour pointer are 4-3-1=0 and 4-2-1=1 respectively). This was done, since for differences of less than 30 minutes the hours pointer begins to become very close to the next digit on the clock and it was considered more fair for one hour to be deducted from the error of the hour pointer.

2.1.3 Multi-head model

Finally, a multi-head neural network design was used to handle the problem of predicting time values. This entailed using shared convolutional and dense layers to extract hierarchical characteristics from input data, followed by separate output heads for forecasting hours and minutes. The hours head used a `softmax` activation function and used `categorical cross-entropy loss` to treat the task as a classification issue with 12 classes representing each hour (0,1,...11). Similarly, the minutes head employed a `linear` activation function and a `mean absolute error loss`, treating minute forecasting as a regression problem and employing the same approach as detailed in our prior regression model, but this time only minutes were forecasted. Regarding the specific architecture and parameters of the network, it takes as input grayscale images with a shape of (75, 75, 1) and it consists of three convolutional layers with increasing filter sizes (128, 64, and 32), with a kernel size of (3, 3), each followed by batch normalization, max-pooling with a size of (2, 2) and rectified linear unit (ReLU) activation. The resulting feature maps are flattened and passed through two fully connected layers with 1024 and 512 units, respectively, each followed by ReLU activation and dropout regularization to prevent overfitting (with a dropout rate of 0.4). By utilizing shared layers, the multi-head model is able to proficiently comprehend complex patterns within the input data and learn valuable features that can be applied across all tasks. As a result, training a single neural network with one output for each task is likely to yield superior results compared to training separate networks. Therefore, it was expected that this model would outperform the two previously described models in terms of performance. For calculating the performance of this model the common sense error function that was explained before in this paper was again used.

2.2 Experiments and Results

2.2.1 Classification

The model was trained using the categorical cross-entropy loss function and the Adam optimizer over 500 epochs with a batch size of 64. During both training and validation, the accuracy and loss metrics were recorded. The model improved significantly over the training epochs, with training accuracy increasing over time from 5.35% to a remarkable 99.13% by the end of the training. Similarly, the training loss decreased rapidly from 3.1912 to 0.0290. The validation accuracy improved as well, beginning at 11.28% and reaching 95.52% by the end of the training and the validation loss also decreased to 0.2368. These findings indicate that the model not only learned well from the training data but also generalized well to previously unseen validation data. Finally, the evaluation of the test set produced an accuracy of 95.58%, confirming the model's robustness and generalization ability.

Subsequently, the training accuracy for the model with 72 categories improved noticeably over the 500 epochs, starting at a modest 1.36% and rising to an impressive 98.07% by the end of the training. The training loss decreased also gradually from 4.3293 to 0.0562. Moreover, the validation accuracy improved consistently, starting at 1.49% and ending at 88.99% and the validation loss also decreased to 0.4973. These findings suggest that the model learned effectively from the training data and generalized well to the validation set. The test set evaluation, finally, yielded an accuracy of 88.75%, confirming the model's robustness and generalization capability.

Finally, for our last classification model with the 720 categories, the initial training accuracy was 0.11%, reflecting the difficult nature of the task. However, the model improved over the course of training, reaching a final training accuracy of

65.43%. The training loss decreased slowly from 6.6144 to 0.9789, indicating that the model learned to minimize categorical cross-entropy loss on training data. Moreover, the validation accuracy increased in a similar manner, beginning at 0.14% and ending at 18.68% and the validation loss fell from 6.5811 to 6.0243. Although validation accuracy remained low, the increasing trend indicates that the model generalized better as training progressed. The accuracy of the test set was determined to be 18.19%. This indicates that, while the model can generalize to new data, it still struggles to accurately classify images into the specified categories.
The performance of all the three classification models are shown in figure 5

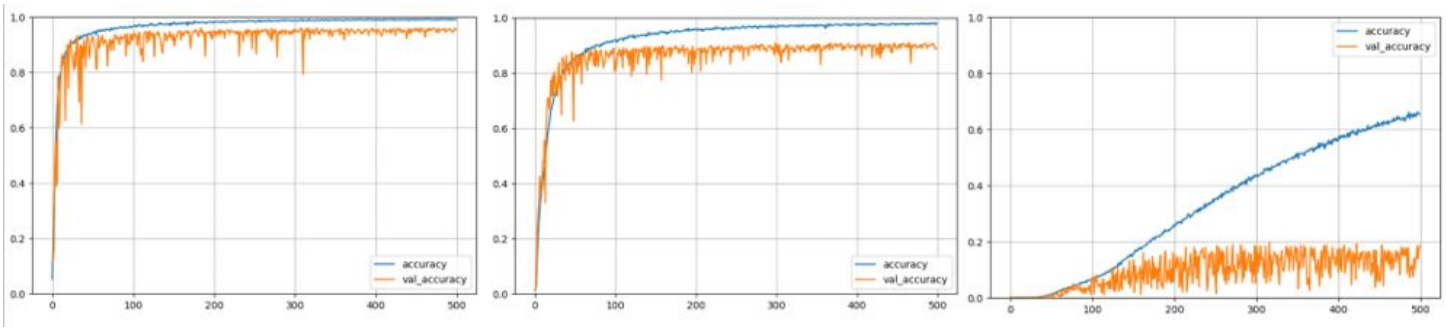


Figure 5: Classification CNNs performance over 500 epochs from left to right: 24, 72, 720 classes

2.2.2 Regression

The regression model was trained for 500 epochs with the Adam optimizer and the mean absolute error (MAE) loss function, with a learning rate of 0.001. As shown on the left of figure 6, the model demonstrated a significant decrease in both training and validation losses throughout the training process, dropping from 2.7769 to 0.2420 and 2.3473 to 0.3098, respectively. These decreasing losses indicate that the model is learning effectively and can generalize well to previously unseen data. The model's proficiency was further confirmed by a low mean absolute error (MAE) of 0.2783 on the test set. This metric indicates that the regression model performed well in accurately predicting continuous time values, demonstrating its superior performance and generalization abilities beyond the training data. Furthermore, the common sense error function, as explained in detail, evaluates the model's error on minutes pointer, on hours pointer as well as the real error in time. So on average, the model exhibited an error of approximately 15.6 minutes. As for the error of the minutes pointer, it was 11.27 minutes and for the hours pointer was 0.10 hours.

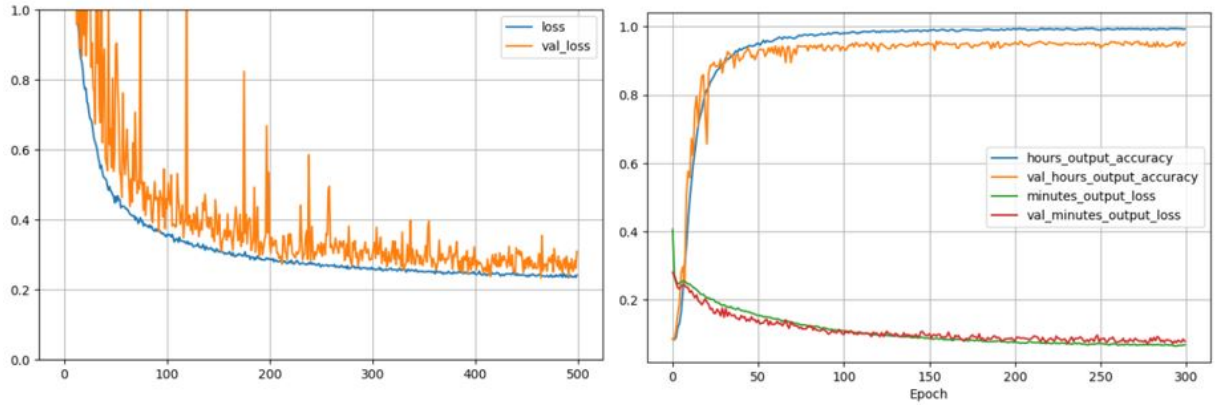


Figure 6: Regression CNN performance over epochs on the left and Multi-Head CNN on the right

2.2.3 Multi-head model

Again, the Adam optimizer with the default learning rate of 0.001 is used to compile the model,and its performance is evaluated based on accuracy for the hours output and mean absolute error for the minutes output. The model was trained for 300 epochs with a batch size of 64, and assessed on the dataset in this configuration for comparison with earlier results. As illustrated on the right of figure 6 the multi-head model had the best performance compared to the other models, with higher accuracy, lower loss and lower common sense error. The loss keeps decreasing, so possibly for more epochs the model might increase its accuracy but this was considered not necessary since even with the 300 epochs it achieved to have the best performance among all the trained networks, with a common sense error of approximately 8 minutes. Regarding the error of

the hours and minutes pointer, it was calculated to be approximately 5 minutes and 0.05 hours respectively which is quite impressive.

Task 3: Generative Models.

The goal of this task is to learn how to train and use generative models for image-related tasks. A Jupyter notebook was provided which illustrates how generative models (CAE, VAE, and GAN) are built and trained. So a different dataset has to be found to preprocess it, retrain these networks, and use the models to generate novel content.

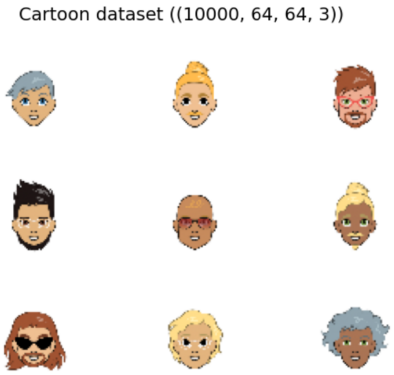
Methodology

3.1.1 Dataset

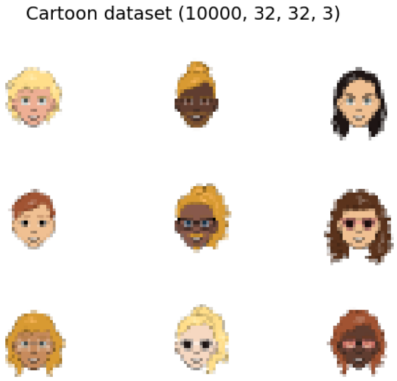
The dataset that was chosen is named 'Cartoon Set' and is a collection of random, 2D cartoon avatar images. The cartoons vary in 10 artwork categories, 4 color categories, and 4 proportion categories. The total number of images that contain the dataset is 10,000. The link to the dataset is: `https://google.github.io/cartoonset/index.html`.

3.1.2 Preprocceess the dataset

The dataset that was selected had first to be processed in order to be compatible with the given file. Thus the images were reshaped from the original 500 x 500 to 64 x 64. Also, a conversion of the dataset from the reshaped images to a numpy array was done. Since the dataset is now compatible with the file provided, it was downsampled from 64 x 64 to 32 x 32. Figure 7a shows an example of 9 cartoons of size 64 x 64 and Figure 7b shows 9 downsampled cartoons of size 32 x 32.



(a) Samples from the dataset with 64 x 64 size



(b) Samples from the dataset with 32 x 32 size

Theory

3.2.1 Autoencoders

Autoencoders are artificial neural networks capable of learning dense representations of the input data, without any supervision. The aim of the autoencoders is to copy their inputs to their outputs. An autoencoder consists of two parts, an encoder and a decoder. The encoder converts the inputs to dense representations, called latent representations while the decoder converts the internal representation to the outputs [2].

3.2.2 Convolutional Autoencoders (CAEs)

Convolutional neural networks are better in image-related tasks, so convolutional autoencoders are suitable for an autoencoder for images. The structure of a CAE is the same as an autoencoder, specifically an encoder followed by a decoder. In this case, the encoder is a CNN composed of convolutional and pooling layers which reduce the spatial dimensionality of the images and increase the depth. On the contrary, the decoder upscales the images and reduces the depth back to the original dimensions [2].

[2]Hands-on machine learning with Scikit-Learn and TensorFlow : concepts, tools, and techniques to build intelligent systems

3.2.3 Variational Autoencoders (VAEs)

Variational autoencoders (VAEs) are probabilistic autoencoders which means that the output is determined by chance. In addition, VAEs are generative autoencoders, as they have the ability to generate new images that look like the input images. The difference between VAEs and other autoencoders is that instead of producing latent representations for an input, the encoder produces a mean coding μ and a standard deviation σ which uses them for Gaussian distribution.

VAE's cost function is composed of two parts. The first is the reconstruction loss which pushes the autoencoder to reproduce its inputs. The second is the latent loss which is the divergence between the target distribution and the actual distribution of the codings [2]. Equation 2 shows the VAE's latent loss.

$$\mathcal{L}_{\text{latent}} = -\frac{1}{2}\sum_{i=1}^{N}\left(1 + \log(\sigma_i^2) - \mu_i^2 - \sigma_i^2\right) \tag{2}$$

Where:
L is the latent loss,
n is the codings' dimensionality,
μ_i is the mean of the i^{th} component of the codings,
σ_i is the standard deviation of the i^{th} component of the codings

3.2.4 Generative Adversarial Networks (GANs)

Generative adversarial networks (GANs) are neural networks capable of generating data. A GAN is composed of two neural networks, the generator and the discriminator. The main idea of GAN is that the generator competes with the discriminator. The generator can be proverbed as the decoder in a VAE. Specifically, the generator tries to produce images that are very similar to the inputs. In comparison, the goal of the separator is to distinguish from the images received from the generator which images are real and which are fake. Also, GANs because they consist of two competing neural networks, have a special way of training. Each training iteration is divided into two phases. In the first phase, only the discriminator is trained and in the second phase, only the generator is trained [2]. Thus the generator learns to make images to fool the discriminator, while the discriminator is trained to learn to distinguish the generator's fake images. The GAN's loss function is binary cross-entropy.

Experiments and Results

3.3.1 CAE

CAE uses the encoder and the decoder with the aim of reconstructing images. The hyperparameters used for the training of the CAE are:
`latent_dim = 1024, num_filters = 256, epochs=300, batch_size=128`. Figure 8 shows nine reconstructed images after the training of 300 epochs.

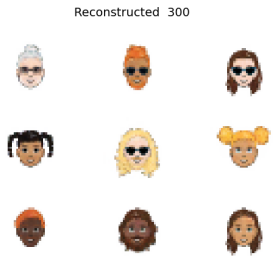
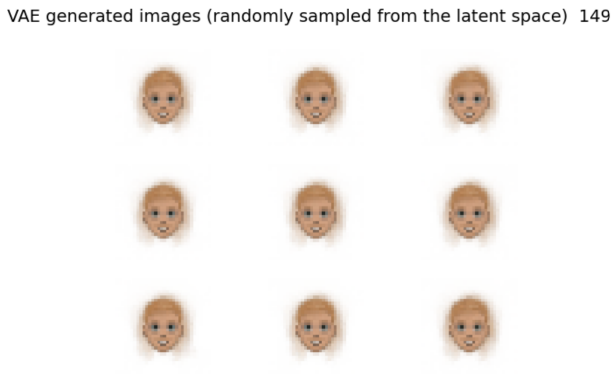


Figure 8: Reconstructed images

3.3.2 VAE

VAE uses the same encoder and decoder as the CAE. The hyperparameters used for the training of VAE are:
`latent_dim = 1024, filters=256, epochs=150, batch_size=128`. Figure 9a shows the results after the training of 150 epochs. The loss in the 300^{th} epoch is: `loss=0.2065`. Although the loss is low, the results are not very good. More specifically, the images take the shape of the face, some features such as eyes, mouth, and nose are clearly formed. However, the model is unable to generate hair or give different colors to the features which leads to a lack of variety between the images as they are identical.

After the training, two randomly generated vectors that represent two points in the latent space are selected as input to the decoder, in order to create linear interpolation to see how the network output changes. The result of the linear interpolation is shown in Figure 9b. Specifically, the Figure shows the blending between two generated samples as you traverse the feature space. However, the results are the same because the results of the training were not the desired ones.



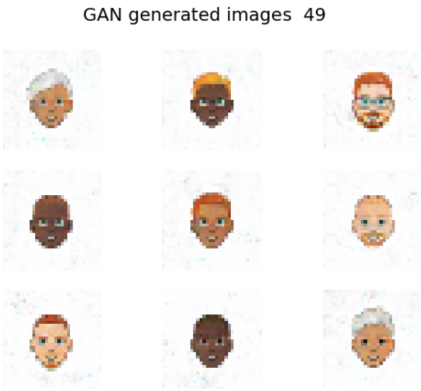
(a) Nine images after VAE'S training of 150 epochs

(b) Linear Interpolation between two random points of latent space

3.3.3 GAN

As mentioned before (section 3.2.4), the training of a GAN is divided into two phases. At each iteration, the discriminator is trained first and then follows the generator's training. The hyperparameters that are used for the training of the GAN model are: `latent_dim = 256, filters=256, n_epochs=50, batch_size=64`. From the 7^{th} epochs onwards, some good results started to appear. Figure 10a shows the results of the training at 7 epochs while Figure 10b presents the results at the end of the training (50 epochs).

(a) Results of the training at 7 epochs



(b) Results of the training at 50 epochs

Similar to the VAE, after the training, two random vectors that represent random points in the latent space are selected as input to the generator, in order to create linear interpolation. The results of two examples of linear interpolation are shown in Figure 11. The left cartoon is the one randomly chosen point and the other random point is the right cartoon. The three images among the two random points show the smooth transition from one point to another. In these two examples can be seen the changes in coloring of the skin, hair, and beard, as well as the addition of accessories such as sunglasses to the right figure. In comparison, the results of both training and linear interpolation between two random points from the latent space of each model, are better in GAN than in VAE in this dataset.

Figure 11: Two examples of linear interpolation

Contributions

Myriana: Task 1: code and report, Task2c & common sense error: code and report
Pelagia: Task 2a, 2b: code and report
Elena: Task 3: code and report